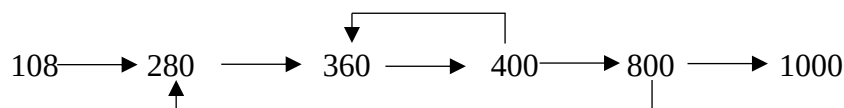


SOLUTION SET

Q1. Version 3A

The memory of a computer is byte-addressable, and the word length is 8 bits. A program consists of a pair of nested loops; a small inner loop (executed 20 times for each outer loop iteration) inside a larger outer loop (executed 10 times). The general structure of the program is given in the figure below. The decimal memory addresses shown delineate the location of the two loops and the beginning and end of the total program. All memory locations in the various sections of the program addresses, 108-279, 280-359, 360-400, 401-800, 801-1000 contain instructions to be executed in straight-line sequencing. Instructions at addresses 400 and 800 are conditional jumps. The program is to be run on a computer that has an instruction cache of 1 KB capacity, organized in the direct-mapped manner with a block size of 16 B. The miss penalty in the instruction cache is $80x$ clock cycles, where x is the hit time of the cache. Assume the cache is initially empty, Compute the miss-rate of the cache and the total time needed for instruction fetching in the execution of the entire program.



Solution: We divide the program into 5 regions

Region-A: 108-279 – 172 instructions, executed only once. (total - 172)

Region B: 280-359 – 80 instructions, executed 10 times. (total - 800)

Region C: 360-400 – 41 instructions, executed 200 times. (total - 8200)

Region D: 401-800 – 400 instructions, executed 10 times. (total - 4000)

Region E: 801-1000 – 200 instructions, executed only once. (total - 200)

All these 893 instructions (each one byte) can be fully contained in a 1KB cache easily. After the compulsory miss, the fetched block will reside in the cache till the end of the program. All subsequent references instructions from such blocks are hits.

Number of sets: = Cache Size/(Block Size x Associativity) = $2^{10} / (2^4 \cdot 1) = 2^6 \rightarrow 64$ bits

Tag bits	Set Index (6-bits)	Block Offset (4-bits)
----------	--------------------	-----------------------

The sets are numbered as S0 to S63 (64 sets). Address 108 (first instruction of the program) is mapped to S6 (108/16 byte) and Address 1000 (last instruction of the program) is mapped to S62 (1000/16 byte). Accordingly, other instructions will occupy from S6 to S62. First instruction from each set will be a miss. We have total 57 misses like that.

Miss_rate = number of misses/numbers of memory access = $57/13372 = 0.00426$ (5 marks)

AMAT = Hit_time + Miss_rate X Miss_penalty = $x + 0.0043 \times 80x = 1.34101x$

Total time needed for instruction fetching = AMAT X IC = $1.34101x \times 13372 = 17931x$ (5 marks)

Another method: (minor variation in answer)

13315 instructions are hits $\rightarrow$ time for fetching = $13315x$

57 instructions are miss $\rightarrow$ time for fetching = $57(x + 80x)$

Total time = $13315x + 57(x + 80x) = 17932x$

Q1. Version 3B

A system has a memory hierarchy that consists of one level of split cache and 1MB of main memory. Both I-cache and D-cache are 4KB direct mapped with 64 byte blocks. The system uses RISC architecture with 32-bit instructions and 32-bit data words. A program that does an array add operation $C[i] = A[i] + B[i]$ is stored in main memory beginning from address 0x42478. A, B and C are one dimensional arrays, each of size 64 data words. Elements are accessed in order from 0th element to 63rd element. The program consists of 10 instructions, out of which last 6 instructions are part of a loop. Each iteration of the loop access one unique element of A, B and C exactly once. The address allocation of A, B and C starts from physical addresses 0x27240, 0x4A2E0 and 0x27A20, respectively. Initially all the caches are empty. Assume all other temporary data variables used by the program, are in registers and are not affecting the cache memory mappings.

- Find the I-cache and D-cache miss rates.
- Find the average MPI (Misses per Instruction) of the program.
- Indicate non-empty sets in I-cache and D-cache at the end of the execution.
- Which all elements of A, B and C are residing in the cache at the end of execution of the program?

Solution: Main memory = 1MB = 2^{20}

L1 I cache: 4KB, 64B block size, direct-mapped.

L1 D cache: 4KB, 64B block size, direct-mapped.

RISC architecture of 32-bit instruction and 32-bit data words.

Each instruction size is of 4B and each data words is of size 4B.

Number of sets: = Cache Size/(Block Size x Associativity) = $2^{12} / (2^6 \cdot 1) = 2^6 \rightarrow 6$ bits

Tag (8-bits)	Set Index (6-bits)	Block Offset (6-bits) Word = first 4 bits
--------------	--------------------	--

Add_Array consists of 10 instructions beginning from address 0x42478 (I0-I9)

0x42478 = 0100 0010 0100 0111 1000 = Set 17 (2-instructions in Set 17 [I0 & I1])

8-instructions (I2-I9) in **Set 18**. But all are hits except I0 and I2 (compulsory misses).

I4 to I9 is iterated 64 times, but always hit.

Address allocation for arrays A, B, and C.

A = 0x27240 = 0010 0111 0010 0100 0000 = Set 9

B = 0x4A2E0 = 0100 1010 0010 1110 0000 = Set 11

C = 0x27A20 = 0010 0111 1010 0010 0000 = Set 40

A array: 64-data words. Since each word is of size: 4B and block size: 64B, Therefore in each block 16 words are brought from main memory.

For A address, the word offset is 0000. Hence, next 4 block will bring the complete array into the memory. These blocks will map to Set number: 9, 10, 11, 12

For B-array, the word offset is 1000. Hence, 1st 8-words will be in set 11, next 16 words in set 12, 13, then 14, and last 8-words in set 15.

For C-array, the word offset is 1000. Hence, Set 40, 41, 42, 43, 44 are brought in cache similar to B.

Array offset	Array A	Array offset	Array B	Array offset	Array C
0-15	Set 9	0-7	Set 11	0-7	Set 40
16-31	Set 10	8-23	Set 12	8-23	Set 41
32-47	Set 11	24-39	Set 13	24-39	Set 42
48-64	Set 12	40-55	Set 14	40-55	Set 43
		56-63	Set 15	56-63	Set 44

- (a) Find the I-cache and D-cache miss rates. **(2 marks)**

Total number of instructions fetched from I-cache = $4 + (6 \times 64) = 388$ [1 miss each @ I0, I2]

Total number of elements accessed from D-cache = $64 \times 3 = 192$.

14 misses: A0, A16, A32, A48, B0, B8, B24, B40, B56, C0, C8, C24, C40, C56 (all compulsory misses)

I-cache miss rate: Number of misses/ totals I-cache access = $2/388 = 0.005$

D-cache miss rate: Number of misses/ total cache access = $14/192 = 0.073$

- (b) Find the average MPI (Misses per Instruction) of the program. **(2 mark)**

16 misses in total for 388 instructions. $MPI = 16/388 = 0.0412$

- (c) Indicate non-empty sets in I-cache and D-cache at the end of the execution. **(3 marks)**

I cache : Sets 17, 18

D-Cache: Sets 9, 10, 11, 12, 13, 14, 15, 40, 41, 42, 43, 44 (total 12 sets)

- (d) Which all elements of A, B and C are residing in the cache at the end of execution of the program?
A0 to A63 (full), B24-B63, C0-C63 (full) **(3 marks)**

Q1. Version 3C

An embedded system unit has a processor, a 64 KB cache memory and a 1 MB main memory. The word length is 64 bits. The cache memory is direct mapped with a block size of 256 words. Operating system kernel code is stored from word 0 to word 4095 (in decimal) in the main memory. The kernel code is in an infinite loop from word 2 to word 4092 (both inclusive). Execution of the code within this loop is fully sequential without any branch or loop instructions except two Interrupt Service Routine (ISR) calls. These ISR calls are at word 2000 (ISR_A) and 4000 (ISR_B). Both these ISRs are 1024 words each and are loaded from the word addresses 8192 and 10240, respectively. The last line of each of the ISR is a return statement back to the kernel. Each ISR code also is fully sequential without any branch or loop instructions. Assume caches are initially empty. All data of kernel and ISRs are in processor registers and will not affect cache mapping.

- (a) Show the physical address split up of tag, set-index and byte offset.
 (b) How many cache sets will be still empty after 50 iterations of the kernel loop? List them.
 (c) If each context switch (control transfer from kernel to ISR/vice versa) takes 'a' milliseconds, execution of an instruction word takes 'b' milliseconds if it is a cache hit or 'c' milliseconds if it is a cache miss, then what is the execution time of 10 iterations of the kernel loop?

Solution:

- (a) Show the physical address split up of tag, set-index and byte offset. **(3 marks)**

Solution: Main memory = 1MB = 2^{20} Bytes (20 bit physical address)

1 word = 64 bits = 8 Bytes, 1 cache block = 256 words = $256 \times 8B = 2048 B$

Cache: 64KB, 2048B block size, direct-mapped.

Number of sets: = Cache Size/(Block Size x Associativity) = $2^{16} / (2^{11} \cdot 1) = 2^5 \rightarrow 5 \text{ bits}$

Tag (4-bits)	Set Index (5-bits) Total 32 sets	Block Offset (11-bits) 8 (word) + 3 (byte in word)
---------------------	---	---

(b) How many cache sets will be still empty after 50 iterations of the kernel loop? List them. Each block is 256 words. Kernel code (4096 word) will take 16 blocks in main memory and they are stored in blocks 0, 1, 2,..., 15 of main memory. These blocks are mapped to 16 sets in cache (S0-S15). [(Main memory block #)%32.]

ISR_A stored from word 8192 to 9215 are in blocks 32, 33, 34, & 35 in main memory. Hence, they mapped to 4 sets (S0, S1, S2, & S3 in cache). [(Main memory block #)%32.]

ISR_B stored from word 10240 to 11264 are in blocks 40, 41, 42, & 43 in main memory. Hence, they are mapped to 4 sets (S8, S9, S10, & S11 in cache).

Kernel		ISRs
S0	→	ISR_A S0
S1		S1
S2		S2
S3		S3
S4 (Not Evicted Out)	→	
S5 (Not Evicted Out)		
S6 (Not Evicted Out)	←	
S7 (WORD 2000)		
S8	→	ISR_B S8
S9		S9
S10		S10
S11		S11
S12 (Not Evicted Out)		
S13 (Not Evicted Out)	→	
S14 (Not Evicted Out)		
S15 (WORD 4000) (Not Evicted Out)	←	
S16-S31 EMPTY		

Hence kernel & ISR_A have 4 cache sets that conflict. Similarly kernel and ISR_B have 4 cache sets that conflict. This will lead to conflict miss and subsequent mutual evictions between blocks of them.

Sets 16 to 31 of cache are empty.

Kernel, ISR_A and ISR_B are restricted to S0 to S15 only. **(3 marks)**

(c) If each context switch (control transfer from kernel to ISR/vice versa) takes 'a' milliseconds, execution of an instruction word takes 'b' milliseconds if it is a cache hit or 'c' milliseconds if it is a cache miss, then what is the execution time of 10 iterations of the kernel loop?

For first iteration, word 2 is already a hit as we brought it at the time of fetching of word 0.

We get 15 misses in kernel (first word of each of the remaining 15 blocks) + 4 misses each in ISR_A and ISR_B respectively (first word of each of the 4 blocks of the respective ISR)

→ $15 + 8 = 23$ misses in first iteration.

In second iteration onwards, we get a miss on the first word that is to be fetched from sets S0, S1, S2, S3, S8, S9, S10, S11 of kernel (total 8 miss in kernel code) + first word that is to be fetched from sets S0, S1, S2, S3 of ISR_A (4 misses in ISR_A) and first word that is to be fetched from sets S8, S9, S10, S11 of ISR_B (4 miss in ISR_B). Whereas fetching from other sets (S4, S5, S6, S7, S12, S13, S14, S15) of kernel code are fully hit, as we have not evicted out their contents.

So, we have total of 16 misses per iteration → $9 \times 16 = 144$ misses in the remaining iterations.

Total number of instruction fetch that are misses = $23 + 144 = 167$. (2 marks)

Total instructions fetched from for 10 iterations of loop = $10 \times (4091 + 1024 + 1024) = 61390$.

Total number of instruction fetch that are hits = $61390 - 167 = 61223$ (2 marks)

We have total 4 context switching (kernel to ISR_A, ISR_A to kernel, kernel to ISR_B, ISR_B to kernel) per iteration → **40 context switching for 10 iterations. (1 mark)**

Execution time = $40a + 61223b + 167c$.

Q1. Version 3D

A program stored in a 16MB main memory system consists of a main routine M which in turn calls two subroutines; P and Q. A processor with a word length as well as instruction length of 16 bits fetches and executes the above program and its associated data. M has 12 instructions (M1-M12) stored continuously from the main memory address 0x78A4FC, P has 4 instructions (P1-P4) stored continuously from the main memory address 0x222454 and Q has 6 instructions (Q1-Q6) stored continuously from the main memory address 0xFCE4F8. M5 to M12 (both inclusive) is a loop that iterates 20 times. M6 is a subroutine call to P and M10 is a subroutine call to Q. The last instruction of both P and Q are subroutine returns back to M. The M, P and Q together access three 4-word integer arrays (F1, F2 and F3) that are assigned physical addresses $F1 \rightarrow 0x400500$, $F2 \rightarrow 0x411500$ and $F3 \rightarrow 0x444600$, respectively. The processor has a 16KB direct mapped unified cache with a block size of 256B. Assume the cache is initially empty. Ignore OS level interruption and subsequent cache impact on context switching.

- Give the split of tag, set index and offset fields of physical address.
- How many compulsory misses and conflict misses occurred during the execution of the program?
- List out the cache block numbers (in decimal) that are non-empty after the execution of the program. Which all instructions of M, P and Q are residing in the cache at the end of execution of the program?
- What is the MPKI value (misses per kilo instruction) of the program?

Solution:

- split of tag, set index and offset fields of physical address. **(2 marks)**

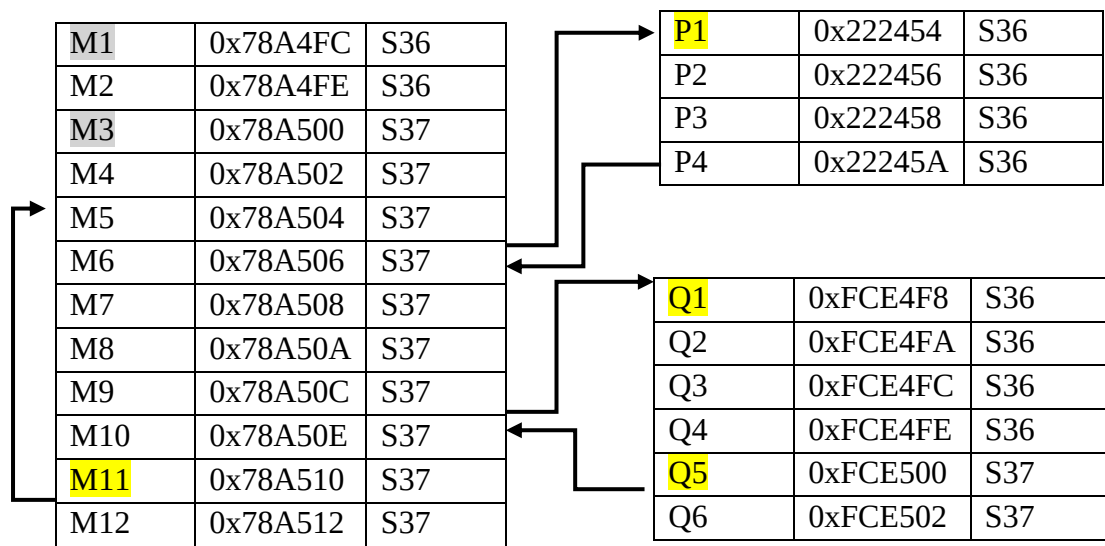
Solution: Main memory = 16 MB = 2^{24} Bytes (24 bit physical address)

1 word = 16 bits = 2 Bytes, 1 cache block = 256 Byte → 128 words.

Cache: 16KB, 256B, direct-mapped.

Number of sets = $\text{Cache Size} / (\text{Block Size} \times \text{Associativity}) = 2^{14} / (2^8 \cdot 1) = 2^6 \rightarrow 64 \text{ sets}$

Tag (10-bits)	Set Index (6-bits) Total 64 sets	Block Offset (8-bits) 7 (word) + 1 (byte in word)
----------------------	---	--



0x78A47C → A4 → 10100100 Index= 36

0x222454 → 24 → 00100100 Index= 36

0x78A500 → A5 → 10100101 Index=37

0xFCE4F8 → E4 → 11100100 Index= 36

0xFCE500 → E5 → 11100101 Index= 37

Data word mapping. Each Array has 4 words, all words part of the same block.

F1→0x400500, 00000101 → Mapped to S5

F2→0x411500, 00010101 → Mapped to S21

F3→0x444600, 01000110 → Mapped to S6

M is mapped to S36 and S37. P is mapped to S36. Q is mapped to S36 and S37.

- (b) There will be compulsory miss on M1(S36), M3(S37), P1(S36), Q1(S36), Q5(S37) and conflict miss on M11(S37) in the first iteration.

Thereafter, for each iteration there will be conflict miss on P1(S36), Q1(S36), Q5(S37), M11(S37).

For data there will be 3 compulsory misses (F1, F2 and F3)

Total 8 compulsory misses and $1 + (19 \times 4) = 77$ conflict misses. **(4 marks)**

- (c) Sets (blocks) S5, S6, S21, S36, S37 are non-empty.

At the end of execution M3 to M12 are in cache (S37), Q1 to Q4 are in cache (S36).

None of the instructions of P are in cache. **(2 marks)**

- (d) Total instructions calculations

M1 to M4 (1 time) = 4 instructions

M5 to M12 (20 times) = 160 instructions

P1 to P4 (20 times) = 80 instructions

Q1 to Q6 (20 times) = 120 instructions

Total = 364 instructions.

857 misses for 364 instructions : MPKI = $(85/364) \times 1000 = 233.5$ **(2 marks)**

SOLUTION SET

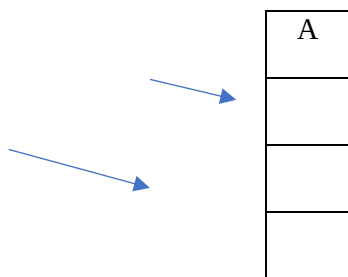
Q2. Version 3A

Consider a 4-way set associative cache that uses pseudo LRU block replacement policy. Assume all the cache blocks are initially empty and filling up of empty blocks in a given cache set happens from way-0 to way-3. Consider the following 13 block numbers (excluding #, \$, and @) all mapped to set n given in the order of arrival. A, R, S, R, B, A, D, C, S, A, S, B, E, #, \$, @.

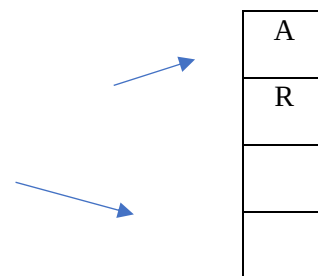
- Find the golden miss ratio of set n, if it is defined as the ratio of compulsory miss to conflict miss in the above sequence of 13 block requests.
- Consider the 14th, 15th and 16th requests #, \$, and @, respectively given above mapped to set n. If access to # resulted in a conflict miss, and access to @ resulted in replacement of block 'S', give the set of all possible values that # and \$ can have such that request to \$ is a hit.

Solution: Reference String for block 'n': A, R, S, R, B, A, D, C, S, A, S, B, E, #, \$, @

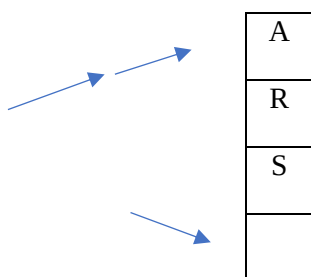
Trees corresponding after each reference:



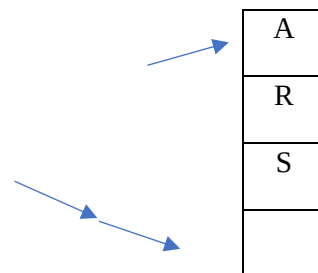
1. Comp. Miss (A)



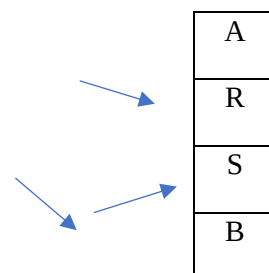
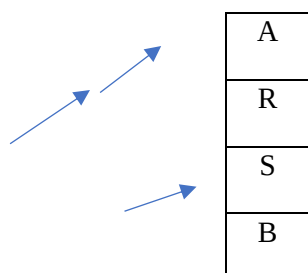
2. Comp. Miss (R)

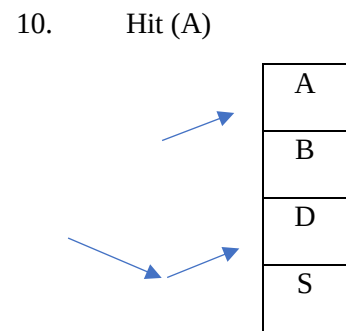
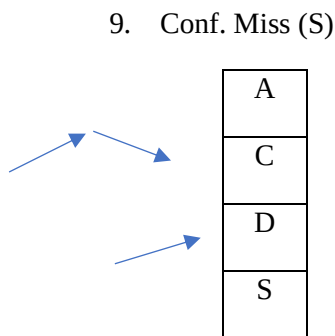
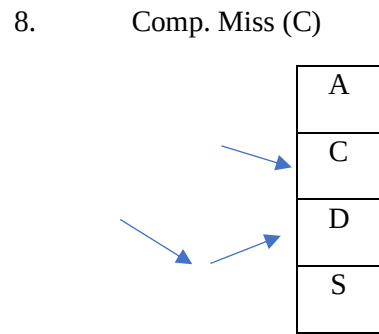
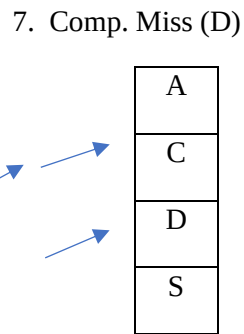
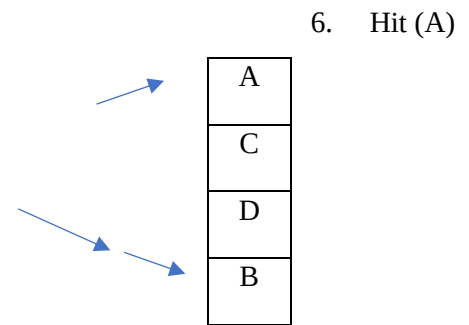
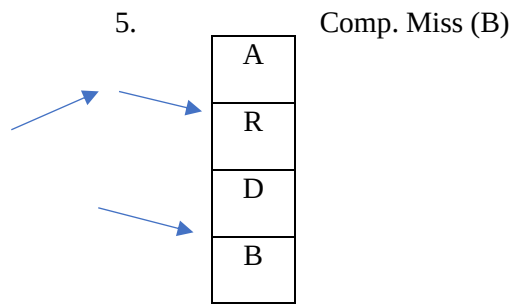


3. Comp. Miss (S)



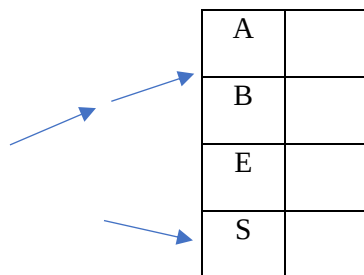
4. Hit (R)





11. Hit (S)

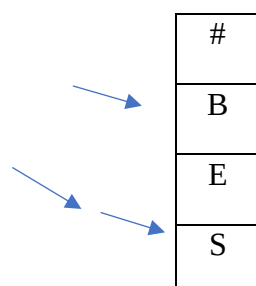
12. Conf. Miss (B)



13. Comp. Miss (E) [Final PLRU tree after serving all 13 request]

(a) Golden ratio = (# compulsory miss/ #conflict misses) = $7/2 = 3.5$ (4 marks)

(b) We know that # results into conflict miss, then resulting tree will be



We know that discrete blocks are (w.r.t. 13 requests) = A, B, C, D, E, R, S

So, **possibilities for # are: C, D, R** (because A has to be replaced for conflict miss).

Now if @ will replace S, then \$ should be a hit in the way-0 and way-1 only to preserve the pointer.

So, possibilities for \$ = #, B ; (Total 6 possibilities)

Hence (#, \$) = { (C, B) (D, B) (R, B) (C, C) (D, D) (R, R) } (4 marks)

Q2. Version 3B

Consider a 4-way set associative cache that uses pseudo LRU block replacement policy. Assume all the cache blocks are initially empty and filling up of empty blocks in a given cache set happens from way-0 to way-3. Consider the following 13 block numbers (excluding $x1$, $x2$, and $x3$) all mapped to set n given in the order of arrival. P, U, W, U, Q, P, S, R, W, P, W, Q, T, $x1$, $x2$, $x3$.

(a) Find the pseudo LRU subtree after processing block number T.

(b) Consider the 14th, 15th and 16th requests $x1$, $x2$, and $x3$, respectively given above mapped to set n. If access to $x1$ resulted in a conflict miss, and access to $x3$ resulted in replacement of block 'W', give the set of all possible values that $x1$ and $x2$ can have, such that request to $x2$ is a hit.

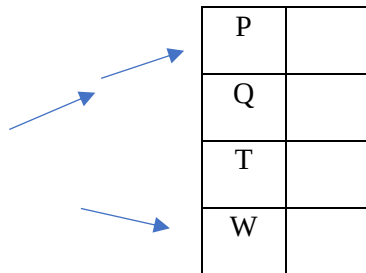
Solution:

In the above question Q2@Version 3A, the string given is

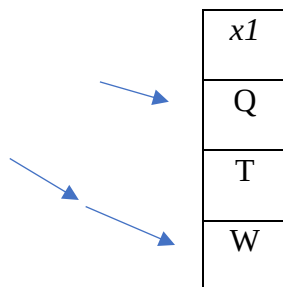
A, R, S, R, B, A, D, C, S, A, S, B, E, #, \$, @. **Here its is**

P, U, W, U, Q, P, S, R, W, P, W, Q, T, $x1$, $x2$, $x3$.

(a) PLRU tree after processing T is as follows (4 marks)



(b) We know that $x1$ results into conflict miss, then resulting tree will be



We know that discrete blocks are (w.r.t. 13 requests) = P, U, W, Q, R, S, T

So, **possibilities for $x1$ are: R, S, U** (because P has to be replaced for conflict miss).

Now if $x3$ will replace W, then $x2$ should be a hit in the way-0 and way-1 only to preserve the pointer.

So, possibilities for $x2 = x1, Q$; (Total 6 possibilities)

Hence $(x1, x2) = \{ (R, Q) (S, Q) (U, Q) (R, R) (S, S) (U, U) \}$ (4 marks)

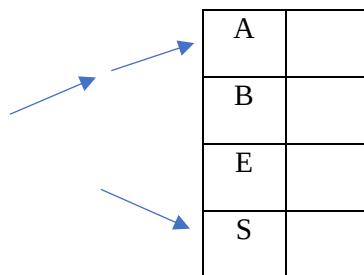
Q2. Version 3C

Consider a 4-way set associative cache that uses pseudo LRU block replacement policy. Assume all the cache blocks are initially empty and filling up of empty blocks in a given cache set happens from way-0 to way-3. Consider the following 13 block numbers all mapped to set n given in the order of arrival. A, R, S, R, B, A, D, C, S, A, S, B, E.

- Find the pseudo LRU subtree after processing these 13 blocks.
- How many conflict misses are there?
- Which of the 4 ways (way-0 to way-3) has maximum cache block replacement during processing these requests?

Solution: Refer to Q2, 3A

- pseudo LRU subtree after processing these 13 blocks. (4 marks)



- How many conflict misses = 2 (2 marks)
- Which of the 4 ways (way-0 to way-3) has maximum cache block replacement during processing these requests? (2 marks)

There are 0 replacement in way-0, 2 replacement in way-1, 2 replacement in way-2, and 1 replacement in way-3. Hence max in way1 and way2

Q2. Version 3D

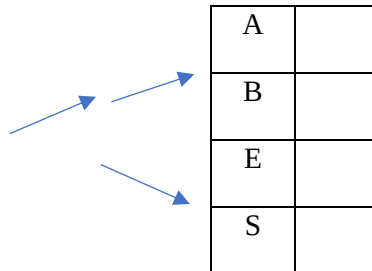
Consider a 4-way set associative cache that uses pseudo LRU block replacement policy. Assume all the cache blocks are initially empty and filling up of empty blocks in a given cache set happens from way-0 to way-3. Consider the following 13 block numbers (excluding #, \$, and @) all mapped to set n given in the order of arrival. A, R, S, R, B, A, D, C, S, A, S, B, E, #, \$, @.

- How many of these 13 requests resulted in hit?
- Consider the 14th, 15th and 16th requests #, \$, and @, respectively given above mapped to set n. If access to # resulted in a conflict miss, and access to @ resulted in replacement of block 'S', give the set of all possible values that # and \$ can have such that request to \$ is a hit.

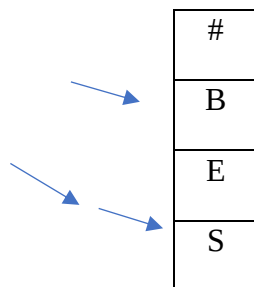
Solution: Refer to Q2, 3A [Same]

(a) How many of these 13 requests resulted in hit? – 4
 Drawing Tree is mandatory to get answers. **(4 marks)**

(b) pseudo LRU subtree after processing these 13 blocks.



We know that # results into conflict miss, then resulting tree will be



We know that discrete blocks are (w.r.t. 13 requests) = A, B, C, D, E, R, S

So, **possibilities for # are: C, D, R** (because A has to be replaced for conflict miss).

Now if @ will replace S, then \$ should be a hit in the way-0 and way-1 only to preserve the pointer.

So, possibilities for \$ = #, B ; (Total 6 possibilities)

Hence (#, \$) = { (C, B) (D, B) (R, B) (C, C) (D, D) (R, R) } **(4 marks)**

SOLUTION SET

Q3. Version 3A

Consider a dynamically scheduled single-issue pipeline with speculation using a branch predictor that always predicts the branch as taken. **ROB has a capacity of 6 entry**. The pipeline has one floating point unit, one memory unit (L.D /S.D), and one integer unit. Branch is also scheduled in the integer unit and branch outcome is written to CDB to initiate committing in ROB. The floating-point unit takes **4 cycles for multiplication and 2 cycles for addition** and they are fully pipelined with normal operand forwarding. Even though the floating-point unit is capable of doing two operations, it can initiate only one operation in a given clock cycle. The integer unit takes 1 cycle and supports full operand forwarding. Memory access of a load operation is done in the immediate cycle after address computation. Stores access memory only upon commit. Assume you have **enough reservation station entries**. At most one CDB write can happen in a cycle.

```

      L.D    F1, 8 (R0)
loop: L.D    F2, 0 (Rx)
      MUL.D  F3, F2, F1
      ADD.D  F5, F5, F3
      S.D    F5, 0 (Ry)
      SUB    Ry, Ry, #8
      SUB    Rx, Rx, #8
      BNEZ   Rx, loop
    
```

Fill up the location of operand, time of issue, execution, MEM access, CDB write and commit for 2 iterations of the loop for the code given above. (Use the following table format)

Itr No	Instruction	Operand 1	Operand2	Issue	Execute	MEM Access	CDB Write	Commit
1	L.D F1, 8 (R0)	Imm	RF	1	2	3	4	5
1	L.D F2, 0 (Rx)	Imm	RF	2	3	4	5	6
1	MUL.D F3, F2, F1	CDB	CDB	3	6		10	11
1	ADD.D F5, F5, F3	RF	CDB	4	11		13	14
1	S.D F5, 0 (Ry)	Imm	RF	5	6	15		15
1	SUB Ry, Ry, #8	RF	Imm	6	7		8	16
1	SUB Rx, Rx, #8	RF	Imm	7	8		9	17
1	BNEZ Rx, loop	CDB	-	8	10		11	18
2	L.D F2, 0 (Rx)	Imm	ROB	12	13	14	15	19
2	MUL.D F3, F2, F1	ROB/CDB	RF	15	16		20	21
2	ADD.D F5, F5, F3	RF	CDB	16	21		23	24
2	S.D F5, 0 (Ry)	Imm	RF	17	18	25		25
2	SUB Ry, Ry, #8	RF	Imm	18	19		20/21*	26
2	SUB Rx, Rx, #8	RF	Imm	19	21		22	27
2	BNEZ Rx, loop	ROB	-	20	23		24	28

Q3. Version 3B

Consider a dynamically scheduled single-issue pipeline with speculation using a branch predictor that always predicts the branch as taken. **ROB has a capacity of 6 entry**. The pipeline has one floating point unit, one memory unit (L.D /S.D), and one integer unit. Branch is also scheduled in the integer unit and branch outcome is written to CDB to initiate committing in ROB. The floating-point unit **takes 6 cycles for multiplication and 2 cycles for addition** and they are fully pipelined with normal operand forwarding. Even though the floating-point unit is capable of doing two operations, it can initiate only one operation in a given clock cycle. The integer unit takes 1 cycle and supports full operand forwarding. Memory access of a load operation is done in the immediate cycle after address computation. Stores access memory only upon commit. Assume you have **enough reservation station entries**. At most one CDB write can happen in a cycle.

```

      L.D    F1, 8 (R0)
loop: L.D    F2, 0 (Rx)
      MUL.D  F3, F2, F1
      ADD.D  F5, F5, F3
      S.D    F5, 0 (Ry)
      SUB    Ry, Ry, #8
      SUB    Rx, Rx, #8
      BNEZ   Rx, loop
  
```

Fill up the location of operand, time of issue, execution, MEM access, CDB write and commit for 2 iterations of the loop for the code given above. (Use the following table format)

Itr No	Instruction	Operand 1	Operand2	Issue	Execute	MEM Access	CDB Write	Commit
1	L.D F1, 8 (R0)	Imm	RF	1	2	3	4	5
1	L.D F2, 0 (Rx)	Imm	RF	2	3	4	5	6
1	MUL.D F3, F2, F1	CDB	CDB	3	6		12	13
1	ADD.D F5, F5, F3	RF	CDB	4	13		15	16
1	S.D F5, 0 (Ry)	Imm	RF	5	6	17		17
1	SUB Ry, Ry, #8	RF	Imm	6	7		8	18
1	SUB Rx, Rx, #8	RF	Imm	7	8		9	19
1	BNEZ Rx, loop	CDB	-	8	10		11	20
2	L.D F2, 0 (Rx)	Imm	ROB	14	15	16	17	21
2	MUL.D F3, F2, F1	ROB/CDB	RF	17	18		24	25
2	ADD.D F5, F5, F3	RF	CDB	18	25		27	28
2	S.D F5, 0 (Ry)	Imm	RF	19	20	29		29
2	SUB Ry, Ry, #8	RF	Imm	20	21		22	30
2	SUB Rx, Rx, #8	RF	Imm	21	22		23	31
2	BNEZ Rx, loop	ROB	-	22	24		25	32

Q3. Version 3C

Consider a dynamically scheduled single-issue pipeline with speculation using a branch predictor that always predicts the branch as taken. **ROB has a capacity of 6 entry**. The pipeline has one floating point unit, one memory unit (L.D /S.D), and one integer unit. Branch is also scheduled in the integer unit and branch outcome is written to CDB to initiate committing in ROB. The floating-point unit takes **4 cycles for multiplication and 2 cycles for addition** and they are fully pipelined with normal operand forwarding. Even though the floating-point unit is capable of doing two operations, it can initiate only one operation in a given clock cycle. The integer unit takes 1 cycle and supports full operand forwarding. Memory access of a load operation is done in the immediate cycle after address computation. Stores access memory only upon commit. Assume you **have 2 reservation station entries** for each functional unit. At most one CDB write can happen in a cycle.

```

L.D    F1, 8 (R0)
loop:  L.D    F2, 0 (Rx)
      MUL.D  F3, F2, F1
      ADD.D  F5, F5, F3
      S.D    F5, 0 (Ry)
      SUB    Ry, Ry, #8
      SUB    Rx, Rx, #8
      BNEZ   Rx, loop
  
```

Fill up the location of operand, time of issue, execution, MEM access, CDB write and commit for 2 iterations of the loop for the code given above. (Use the following table format)
red arrow indicates delay by lack of reservation entries. 

Itr No	Instruction	Operand 1	Operand2	Issue	Execute	MEM Access	CDB Write	Commit
1	L.D F1, 8 (R0)	Imm	RF	1	2	3	4	5
1	L.D F2, 0 (Rx)	Imm	RF	2	3	4	5	6
1	MUL.D F3, F2, F1	CDB	CDB	3	6	10	11	11
1	ADD.D F5, F5, F3	RF	CDB	4	11	13	14	14
1	S.D F5, 0 (Ry)	Imm	RF	5	6	15	15	15
1	SUB Ry, Ry, #8	RF	Imm	6	7	8	16	16
1	SUB Rx, Rx, #8	RF	Imm	7	8	9	17	17
1	BNEZ Rx, loop	CDB	-	9	10	11	18	18
2	L.D F2, 0 (Rx)	Imm	ROB	12	13	14	15	19
2	MUL.D F3, F2, F1	ROB/CDB	RF	15	16	20	21	21
2	ADD.D F5, F5, F3	RF	CDB	16	21	23	24	24
2	S.D F5, 0 (Ry)	Imm	RF	17	18	25	25	25
2	SUB Ry, Ry, #8	RF	Imm	18	19	20/21*	26	26
2	SUB Rx, Rx, #8	RF	Imm	19	21	22	27	27
2	BNEZ Rx, loop	ROB/CDB	-	22	23	24	28	28

Q3. Version 3D

Consider a dynamically scheduled single-issue pipeline with speculation using a branch predictor that always predicts the branch as taken. **ROB has a capacity of 4 entry**. The pipeline has one floating point unit, one memory unit (L.D /S.D), and one integer unit. Branch is also scheduled in the integer unit and branch outcome is written to CDB to initiate committing in ROB. The floating-point unit takes **4 cycles for multiplication and 2 cycles for addition** and they are fully pipelined with normal operand forwarding. Even though the floating-point unit is capable of doing two operations, it can initiate only one operation in a given clock cycle. The integer unit takes 1 cycle and supports full operand forwarding. Memory access of a load operation is done in the immediate cycle after address computation. Stores access memory only upon commit. Assume you have **enough reservation station entries**. At most one CDB write can happen in a cycle.

```

      L.D    F1, 8 (R0)
loop: L.D    F2, 0 (Rx)
      MUL.D  F3, F2, F1
      ADD.D  F5, F5, F3
      S.D    F5, 0 (Ry)
      SUB    Ry, Ry, #8
      SUB    Rx, Rx, #8
      BNEZ   Rx, loop
  
```

Fill up the location of operand, time of issue, execution, MEM access, CDB write and commit for 2 iterations of the loop for the code given above. (Use the following table format)

Itr No	Instruction	Operand 1	Operand2	Issue	Execute	MEM Access	CDB Write	Commit
1	L.D F1, 8 (R0)	Imm	RF	1	2	3	4	5
1	L.D F2, 0 (Rx)	Imm	RF	2	3	4	5	6
1	MUL.D F3, F2, F1	CDB	CDB	3	6		10	11
1	ADD.D F5, F5, F3	RF	CDB	4	11		13	14
1	S.D F5, 0 (Ry)	Imm	RF	6	7	15		15
1	SUB Ry, Ry, #8	RF	Imm	7	8		9	16
1	SUB Rx, Rx, #8	RF	Imm	12	13		14	17
1	BNEZ Rx, loop	ROB	-	15	16		17	18
2	L.D F2, 0 (Rx)	Imm	ROB	16	17	18	19	20
2	MUL.D F3, F2, F1	CDB	RF	17	20		24	25
2	ADD.D F5, F5, F3	RF	CDB	18	25		27	28
2	S.D F5, 0 (Ry)	Imm	RF	19	20	29		29
2	SUB Ry, Ry, #8	RF	Imm	21	22		23	30
2	SUB Rx, Rx, #8	RF	Imm	26	27		28	31
2	BNEZ Rx, loop	ROB	-	29	30		31	32

SOLUTION SET – Q4 for all versions.

Q4. Version 3A

Consider two different cache organizations X and Y that can be attached to a processor. X has 16KB Instruction cache (IC) and 16KB Data cache (DC). Y has 32KB Unified cache (UC) with a single port. Miss rates of IC and DC are 0.64% and 6.47% respectively. Aggregate miss rate of UC is 1.99%. Assume that an application has 33% of data access operations. Hit time is 1 cycle and miss penalty is 50 cycles. By calculating AMAT, decide which organization is better for this application? As a further upgrade, let's consider another organization Z wherein the UC is a dual ported cache, but requires 1.2 cycle hit time. Is Z better? Support your answer.

Solution:

X: 16KB IC + 16 KB D cache

MR_IC = 0.0064, MR_DC=0.0647

Y: 32KB unified cache with 1CC HT + 50CC miss penalty (Additional 1 CC for D cache hits)

MR_UC_uniported=0.0199

Z: 32KB unified cache with 1.2 CC HT + 50CC miss penalty

MR_UC_dualported=0.0199

AMAT = $H_t + MR \times MP$

33% data access →

Cache will be accessed for I in (100/133)% and for D (33/133)%

% time I access = 75.18%, % time D access = 24.82%

Case X:

AMAT = %I access [$H_{t_IC} + MR_{IC} \times MP$] + %D access [$H_{t_DC} + MR_{DC} \times MP$]

= $0.7518[1 + 0.0064 \times 50] + 0.2482[1 + 0.0647 \times 50]$

= $0.992 + 1.05 = \mathbf{2.042 \text{ CC. (1 mark)}}$

Case Y: (Here additional 1CC extra for D cache hits)

AMAT = %I access [$H_{t_UC} + MR_{UC} \times MP$] + %D access [$H_{t_UC} + H_{t_UC} + MR_{UC} \times MP$]

= $0.7518[1 + 0.0199 \times 50] + 0.2482[1 + 1 + 0.0199 \times 50]$

= $1.50 + 0.74 = \mathbf{2.24 \text{ CC. (1 mark)}}$

Case X (split cache organization) is better than Y (unified cache, 1 port) in AMAT. (1 mark)

Case Z: (Here $H_t = 1.2 \text{ CC}$, all other parameters same.)

AMAT = %I access [$H_{t_UC} + MR_{UC} \times MP$] + %D access [$H_{t_UC} + H_{t_UC} + MR_{UC} \times MP$]

= $0.7518[1.22 + 0.0199 \times 50] + 0.2482[1.22 + 0.0199 \times 50]$

= $\mathbf{2.215 \text{ CC.}}$

Z is having lower AMAT than Y but still higher AMAT than X. (1 mark)

Q4. Version 3B

A 32-bit word processor that uses a 32 KB, 4-way, L1 cache and 256 KB, 16-way, L2 cache has a physical address space of 16 MB. The block size of L1 and L2 caches are 16 and 64 words, respectively. Upon an L1 cache miss, it takes 50 ns to return the first word from L2 cache and additional 5 ns each for bringing subsequent words in the same block.

- (a) What is the physical address split of tag, set-index and byte-offset of L1 and L2 caches?
(b) What is the miss penalty of an L1 cache miss occurred for a word whose first byte is at address 0x8645A4, assuming that at L1 cache level the following techniques are implemented? (i) early restart mechanism (ii) critical word first mechanism.

Solution: Address space: 16 MB: Address bits = 24 bits: 32-bit words $\rightarrow$ 1 word = 4 bytes.

Block size = 16 words (L1) & 64 words (L2) (@ 4 bytes per word) = 64B (L1) & 256B (L2), respectively.

(a) Tag index and offset split up of L1 and L2 caches

Size of L1 cache = 32 KB: L1 is 4-way set associative: 16 words $\rightarrow$ 64B block

Number of sets: = Cache Size/(Block Size x Associativity) = $2^{15} / (2^6 \times 2^2) = 2^7 \rightarrow 7$ bits

Total **24 bit address: Tag=11, set-index=7 and offset=6** (4 bit word+ 2 bit byte within word)

(1 mark)

Size of L2 cache = 256 KB: L2 is 16-way set associative: 64 words $\rightarrow$ 256B block

Number of sets: = Cache Size/(Block Size x Associativity) = $2^{18} / (2^8 \times 2^4) = 2^6 \rightarrow 6$ bit

Total **24 bit address: Tag=10, set-index=6 and offset=8** (6 bit word+ 2 bit byte within word)

(1 mark)

(b) Cache Miss penalty calculation

Here we use split up of L1 cache only

Time for bringing first word from L2 = 50 ns

Additional time for subsequent words = 5 ns for each word

A1 address = 0x 8645A4; Converting A1 address into bits

0x 8645A4 = **1000 0110 0100 0101 1010 0100** word offset= **1001** $\rightarrow$ 9 (10th word)

(i) Early Restart Optimization

The missed word is word number 9 (10th word) in the block. As and when this word is fetched processor will resume execution.

Miss Penalty = time for 1st word + 9 x additional time for subsequent word. = 50 + (9 x 5) = **95 ns**

(1 mark)

(ii) Critical Word First Optimization

Here the critical word (10th word) will be brought first.

Miss Penalty = time for 1st word = 50 ns **(1 mark)**

Q4. Version 3C

Consider a cache that has a miss penalty of 200 clock cycles, and all instructions normally take 1 clock cycles for a perfect cache that is 100% hit. It was found that the processor with the perfect cache has 7 times speed up over the realistic cache while running a program that has an average of 1.5 memory references per instruction.

- (a) What is the miss rate of the realistic cache and average MPKI for this program?
(b) How much slowdown (approximate) will the program experiences if there was no cache memory in the memory hierarchy?

Solution:

Mem Access/Instruction = 1.5 : Miss Penalty = 200 CC : CPI_ideal=1

With perfect cache or realistic cache, Instruction Count (IC) and Clock Cycle Time (CCT) are same.

(a) CPU CPU_ideal = IC x CPI_ideal x CCT

CPU time_realistic = IC x {CPI_ideal + Missrate x (Mem Access/Inst) x Miss Penalty} x CC Time

Given that speed up of 7, CPU time_realistic/CPU_ideal = 7

→ $7 = IC \times \{1 + (Mr \times 1.5) \times 200\} \times CCT \rightarrow 6 = 300 \times Mr \times IC \times CCT \rightarrow Mr = 0.02$ (2 marks)

→ Miss rate: (# misses/#mem access) = (miss / instruction)/(mem acc /instruction)

→ MR = MPI/MAPI : MPKI = (MR x MAPI) X 1000 = $0.02 \times 1.5 \times 1000 = 30$ (1 mark)

(b) If no cache memory, then each memory access takes 200 cc.

CPU time_w/o cache = IC x 1 + 1.5 x 200 x CCT = 301 x IC x CCT (~ 300 excluding CPI_ideal)

Speed up = (CPU time_w/o cache/ CPU time_realistic) = $301/7 \sim 43$ times. (1 mark)

Q4. Version 3D [4 marks]

You are given an N-way set-associative write back cache containing a total of T bytes of data, with a block size of B bytes. Assume that N, T and B are integer powers of 2; that is $N=2^n$, $T=2^t$ and $B=2^b$ for some positive integers n, t and b. Consider a 16-bit physical addresses, and assume that a valid bit and minimum LRU bits are used per block. What is the total number of storage bits necessary to implement the cache?

Solution:

Total Cache size: $T=2^t$: Associativity: $N=2^n$: Block Size: $B=2^b$

Number of sets = $CS/(BS \times A) = 2^t / (2^b \times 2^n) = 2^{t-b-n}$

Number of blocks in cache = $CS/(BS) = 2^t / (2^b) = 2^{t-b}$

Bits to represent set number is $t-b-n$.

16 bit physical address: Tag bits = $16 - (t-b-n+b)$

Tag	Index	Offset
$16-t+n$	$t-b-n$	b

For each block there is tag entry + 1-bit dirty bit (WB cache)+ 1 valid bit + n LRU bits

Total control bits per block = $16-t+n + 1 + 1 + n = 18+2n-t$

Total data array = 2^t : Total data bits per block = 2^b

Total storage overhead of control fields (tag array) = number of blocks x $(18+2n-t)$
 $= 2^{t-b} \times (18+2n-t)$

Total overhead for implementing cache = $2^t + 2^{t-b}(18+2n-t)$

Alternative approach:

Bits needed per block = Tag array bits + data array bits = $(18+2n-t) + (2^b)$

Total overhead for implementing cache = # of blocks x bits/block = $2^{t-b}(18+2n-t+2^b)$